

Getting Started with the URL Shortener Project

Web Engineering 2026-2027

Web Engineering Course Team

2026-09-19

Table of contents

1 urlshortener (IW 2627 seed)	1
1.1 Product (minimum expected)	1
1.2 Provided vs grown	2
1.3 Local dev (no Docker)	2
1.4 Quality gates (CI enforced)	2
1.5 Architecture	2
1.5.1 Decisions (required for grown work)	3
1.6 Events	3
1.7 Scale evidence	4
1.8 First 30 minutes	4
1.9 Project Agreement (October 2)	5

1 urlshortener (IW 2627 seed)

This repo is the URL Shortener product seed for 2627 group projects.

1.1 Product (minimum expected)

Graded deliverable: Boot jar(s) **plus** this Compose stack (Postgres + 2 app replicas + LB) — not “jar on localhost only.”

Day-to-day coding uses `./gradlew bootRun` (in-memory HSQLDB). Demo, scale evidence, and grading use Compose:

Layer	Command
Product	<code>docker compose up --build → LB</code> <code>http://localhost:8080</code>
Load (same project)	<code>docker compose --profile load run --rm</code> <code>k6 or host scripts/load.k6.js</code>
Optional broker stub	<code>docker compose --profile broker up</code>

1.2 Provided vs grown

Provided (seed, weight 0):

- Modulith application modules: `links`, `clicks`, `analytics`.
- Seed feature cards (implemented):
 - `0001-create-short-url` — POST `/api/link`
 - `0002-redirect-and-click-log` — GET `/hash` + permanent click log
 - `0003-link-stats` — GET `/api/stats/hash`
- Per-module hexagon with **jMolecules** stereotypes and ArchUnit guards.
- Compose product stack.
- ADRs under `docs/adr/`.
- `AGENTS.md`.
- k6 load script + replica-failover smoke (`scripts/scale-baseline.sh`).

Grown (students):

- New feature cards from `0004-...` (budget 84; 3 features; 2 event-coupled; 5 scored).
- Code in owner modules (`links` / `clicks` / `analytics` or a new module) — keep hexagon + `ApplicationModules.verify()` green.
- Scale evidence as claimed on cards: `replica-failover`, `load-compare`, `cross-instance` (`--profile broker`). Architecture verify (`ModularityTests`) counts toward **engineering** / **ADR**, not scale.
- ADRs under `docs/adr/` when a choice locks modules, events, Level 4, or tech (`0000-TEMPLATE.md`; seed `0001`).
- Specialised technologies (2) and optional Web UI / module extraction.
- Defence packet: cite feature cards, ADRs, and the commands/tests that prove them.

1.3 Local dev (no Docker)

```
./gradlew bootRun      # HSQLDB in-memory
./gradlew test          # Modulith verify + hexagonal ArchUnit + flow test
./gradlew check         # All gates: ktlint + detekt + test + JaCoCo (60%)
./gradlew ktlintFormat  # Auto-fix style issues
```

1.4 Quality gates (CI enforced)

Gate	Tool	Threshold
Code style	ktlint	Zero violations
Static analysis	detekt	Zero issues (see <code>config/detekt.yml</code>)
Coverage	JaCoCo	60% line coverage
Architecture	Modulith + jMolecules	<code>verify()</code> + <code>ensureHexagonal(SEMI_STRICT)</code>

1.5 Architecture

Modulith modules are the **bounded contexts**. Inside each module, use a small hexagon:

Layer	Role
base package	Public API (events other modules may import)
domain	Framework-free model (@Application)
application	Use cases + @PrimaryPort / @SecondaryPort
adapters.*	@PrimaryAdapter / @SecondaryAdapter (web, JPA, events, tech)

`ApplicationModules.verify()` enforces closed-module rules and `jMolecules` hexagon (`ensureHexagonal(SEMI_STRICT)`). Do not reach into another module's internals.

1.5.1 Decisions (required for grown work)

Decide explicitly; do not inherit the seed silently when you change structure or stack.

Decision	Seed default	If you change it
Module boundaries	<code>links</code> / <code>clicks</code> / <code>analytics</code>	New ADR + keep <code>verify()</code> green
Hexagon enforcement	<code>jMolecules</code> stereotypes + <code>ensureHexagonal(SEMI_STRICT)</code> in <code>ModularityTests</code>	ADR (e.g. drop <code>jMolecules</code> , or move to <code>STRICT</code>)
Cross-module API	Events in module base packages	ADR (named interfaces / <code>allowedDependencies</code>)
Async across replicas	In-process only	ADR for Level-4 broker path

Defence packet: list ADRs + the command/test that proves each decision.

1.6 Events

- `links.ShortUrlCreated` → `analytics` creates zeroed `LinkStats`
- `clicks.ClickLogged` (published from redirect in `links`) → `clicks` appends log row; `analytics` increments `LinkStats`

In-process events do **not** cross replicas. Level 4 = externalize via broker (`--profile broker`).

```

flowchart LR
    subgraph links [links module]
        LC[LinkController] --> LS[LinkService]
        LS -->|publish| SUC((ShortUrlCreated))
        LS -->|publish| CL((ClickLogged))
    end
    subgraph clicks [clicks module]
        CL --> CR[ClickRecorder]
    end
    subgraph analytics [analytics module]
        SUC --> LSL[LinkStatsListener]
        CL --> LSL
    end

```

1.7 Scale evidence

Evidence	When required	Artifact	Command
replica-failover	Horizontal claimed	Create/redirect via LB; kill one replica; shared DB still serves	<code>./scripts/scale-baseline.sh</code> then <code>docker compose stop app2</code>
load-compare	Horizontal or weight 13	Same script N=1 vs N=2; RPS + p95 + error rate + interpretation	<code>docker compose --profile load run --rm k6</code>
cross-instance	Level 4 claimed	Side-effect across instances + idempotency	Broker integration + test

Architecture gates (`ModularityTests`, `ensureHexagonal`) belong in **engineering** / **ADRs**, not in this table.

1.8 First 30 minutes

```
# 1. Clone your private repo (fork / copy from this seed)
git clone https://github.com/UNIZAR-30246-WebEngineering/<your-repo>.git
cd <your-repo>

# 2. Check the build passes (ktlint + detekt + test + JaCoCo)
./gradlew check           # all quality gates must be green before you start

# 3. Run locally (HSQLDB in-memory)
./gradlew bootRun         # → http://localhost:8080

# 4. Try the three seed endpoints
http POST localhost:8080/api/link url=https://example.com
http GET  localhost:8080/<hash>      --follow
http GET  localhost:8080/api/stats/<hash>

# 5. Bring up the full product stack (Postgres + 2 replicas + LB)
docker compose up --build  # → LB on http://localhost:8080

# 6. Run the load profile (optional, produces load-compare evidence)
docker compose --profile load run --rm k6
```

Once the stack is up and you can create, redirect, and fetch stats, you are ready to fill your agreement.

1.9 Project Agreement (October 2)

1. **Budget:** 84 total weight, 3 grown features, 2 event-coupled, 5 scored. Weights: $5 \cdot 8 \cdot 13 \cdot 21$ (= ExpectedLevel 1–4). Full rules: [project.pdf](#).
2. **Feature cards:** copy [docs/features/0000-TEMPLATE.md](#) for each scored feature, starting from 0004-...md.
3. **Cover sheet:** fill [AGREEMENT.md](#) — team roster, ownership table, integration owner, budget totals.
4. **Submit:** share your private repo URL with the instructor.